

Onboarding nového developera — projekt Masha

Príprava pred nástupom (.NET + Angular)

Ing. Peter Motyčák, KROS a.s.

2026-05-26

Úvod

Aktuálny stav projektu Masha

Backend (.NET)

Frontend (Masha.WebApp)

Plánované upgrade projekty

Upgrade .NET (rýchlejší)

Upgrade Angularu (náročnejší)

Priorita štúdia

P0 — Základ, bez tohto sa nezaobíde

P0 — Novinky pre .NET upgrade

P1 — KROS-specific stack (verejné repá)

P1 — Frontend stack Masha

P2 — Moderné Angular features (po upgrade ich bude bežne používať)

P2 — Doplnkové (užitočné po nástupe)

P3 — Voliteľné

Praktický cvičný projekt (odporúčaný)

Backend — vlastné API (napr. "Library API")

Frontend — Angular SPA napojená na backend

Skúsiť upgrade na vlastnom projekte

AI nástroje vo vývoji

Hlavné AI nástroje, s ktorými sa stretne

Prečo AI nástroje pri upgrade projektoch

Claude Code — skills a plugins

Odporúčania pre vývojára

Po nástupe (až mu pridáme prístupy)

Odporúčané poradie samotných upgradeov

Zhrnutie kľúčových verejných zdrojov

Úvod

Tento dokument je sprievodca štúdiom pre **nového .NET + Angular developera**, ktorý nastupuje na projekt **Masha**. Predchádzajúca skúsenosť vývojára: **Python** (interné aplikácie v Cenekone). Cieľom je dať mu prehľad o technológiách, s ktorými sa stretne, a o verejných materiáloch, na ktorých sa môže pripraviť **pred podpisom zmluvy**.

Dôležitý kontext k prístupom:

- Prístup k repu Masha, internému kódu, firemnému emailu a interným nuget feedom (`nuget.kros.wtf` , Azure DevOps `krossk/Masha`) dostane **až po podpise zmluvy a pridelení firemného emailu**.
- Pred nástupom má k dispozícii **iba verejné zdroje** — všetky materiály v tomto dokumente sú voľne dostupné.
- Rýchlosť štúdia je individuálna**, dokument neurčuje konkrétny rozvrh.

Aktuálny stav projektu Masha

Backend (.NET)

Vrstva	Technológia	Verzia
Framework	ASP.NET Core	.NET 8
Serverless	Azure Functions (isolated worker, Durable Task)	v2
ORM	Kros.KORM	7.x
CQRS / Mediator	MediatR + Kros.MediatR.Extensions	12
Validácia	FluentValidation	11
Mapping	Mapster	7
HTTP klient	Refit	8
Excel	EPPlus, ExcelDataReader	7
DI scanning	Scrutor	6
Autentifikácia	Kros.Identity.Extensions	0.7
Testy	xUnit, NSubstitute, FluentAssertions	—
Analýzy	Roslynator	4

Štruktúra repa Masha:

- `Masha.Admin` — administračné Web API + funkcie
- `Masha.AzFunImport` — Azure Functions (import dát)

- `Masha.Base` — zdieľaná knižnica
- `Masha.Pipelines` — CI/CD pipelines
- `Masha.Utils` — helpery
- `Masha.WebApp` — frontend (Angular)

Frontend (Masha.WebApp)

Vrstva	Technológia	Verzia
Framework	Angular	15.2 (starý — pôjde upgrade)
State management	NgRx (klasický) — Store, Effects, Entity, Router-Store	15
Forms	ngrx-forms	8
UI knižnica	PrimeNG + PrimeFlex + PrimeIcons	15
Reactive	RxJS	7.5
Polyfills	zone.js	0.11
Jazyk	TypeScript	4.8
Autentifikácia	oidc-client-ts, <code>@kros-sk/auth</code>	2
Realtime	<code>@microsoft/signalr</code>	6
Monitoring	Application Insights	2
Workspace	Angular CLI (NIE Nx)	—
Testy	Karma, Jasmine, Protractor	—

Plánované upgrade projekty

Vývojár sa bude **aktívne podieľať** na dvoch paralelných upgrade projektoch.

Upgrade .NET (rýchlejší)

Vrstva	Z čoho	Na čo
.NET runtime	.NET 8 (LTS do 11/2026)	.NET 10 (LTS) — vyšiel 11/2025
C# language	C# 12	C# 14

Postup: úprava `<TargetFramework>` v `Directory.Build.props`, bump verzií Kros.* knižníc, oprava breaking changes. **.NET 9 sa preskakuje** (je STS, nie LTS).

Upgrade Angularu (náročnejší)

Vrstva	Z čoho	Na čo
Angular	15	20 / 21
NgRx	15	matching s Angular verziou
PrimeNG	15	vyššia (s migráciou na Aura theming)
TypeScript	4.8	5.x
RxJS	7.5	7.8
zone.js	0.11	0.16

Postup: cez všetky major verzie postupne (`ng update`), na každom kroku riešiť breaking changes. Najväčšie skoky sú v15 → 16 (signals preview), v16 → 17 (standalone default, nový control flow, esbuild builder).

Čo zostáva (neuvažuje sa zmena):

- Klasický NgRx (NIE Signal Store)
- PrimeNG (NIE DevExtreme)
- Angular CLI workspace (NIE Nx monorepo)

Priorita štúdia

P0 — Základ, bez tohto sa nezaobíde

Téma	Zdroj
C# 12+ (records, pattern matching, primary constructors, <code>async</code> / <code>await</code> , DI)	Microsoft Learn — C#
.NET 8 základy (hosting, configuration, DI, logging)	Microsoft Learn — .NET
ASP.NET Core Web API (minimal API, controllery, middleware)	Microsoft Learn — ASP.NET Core
TypeScript 5 (strict mode, generiká, utility types, decorators)	typescriptlang.org
Angular 15 (aktuálny stav Masha) — NgModules, components, services	v15.angular.io
RxJS 7 (operátory, Subjects, error handling)	rxjs.dev

P0 — Novinky pre .NET upgrade

Téma	Zdroj
Čo je nové v C# 13	What's new in C# 13
Čo je nové v C# 14	What's new in C# 14
Čo je nové v .NET 10	What's new in .NET 10
Čo je nové v ASP.NET Core 10	ASP.NET Core 10 release notes
.NET migration & breaking changes	.NET compatibility guides

P1 — KROS-specific stack (verejné repá)

Téma	Zdroj
Kros.AspNetCore.BestPractices — referenčná mikroservisná architektúra (CQRS s MediatR, KORM, FluentValidation, Mapster, Scrutor, Ocelot, Redis, Polly). Najlepší zdroj pre pochopenie KROS .NET stacku.	GitHub
Kros.KORM — KROS micro-ORM (alternatíva k EF Core, fluent mapping, bulk operácie)	GitHub
Kros.AspNetCore — KROS utility (ProblemDetails, health checks, middleware)	GitHub
Kros.Utils — univerzálne C# helpery	GitHub
MediatR pattern — Request/Handler, Pipeline behaviors	MediatR GitHub
FluentValidation — validátory, integrácia s MediatR pipeline	docs.fluentvalidation.net
Mapster — entity ↔ DTO mapping	GitHub

P1 — Frontend stack Masha

Téma	Zdroj
NgRx (klasický) — Store, Actions, Reducers, Effects, Selectors, Entity, <code>createFeature</code> , functional effects	ngrx.io
ngrx-forms — form state v store (Masha to používa)	ngrx-forms.readthedocs.io
PrimeNG 15+ — komponenty, theming	primeng.org
Angular upgrade cesta v15 → v20/v21 — každý major step má vlastné breaking changes	update.angular.io

P2 — Moderné Angular features (po upgrade ich bude bežne používať)

Téma	Kde
Standalone components (v15+, default v17+)	angular.dev
Signals (<code>signal</code> , <code>computed</code> , <code>effect</code>) — v16+	angular.dev/guide/signals
Nový control flow (<code>@if</code> , <code>@for</code> , <code>@switch</code> , <code>@defer</code>) — v17+	angular.dev
<code>inject()</code> funkcia namiesto constructor injection	angular.dev
Nový build system (<code>@angular/build</code> , esbuild/Vite)	angular.dev

P2 — Doplnkové (užitočné po nástupe)

Téma	Zdroj
Azure Functions isolated worker (.NET 8+) + Durable Task	Microsoft Learn — Azure Functions
xUnit + NSubstitute + FluentAssertions	xunit.net , nsubstitute.github.io
OIDC / OAuth2 (oidc-client-ts)	oauth.com
Refit — typed HTTP klienti	GitHub
EPPlus — Excel processing	epplussoftware.com
Polly — retry, circuit breaker	GitHub
Flurl — fluent HTTP	flurl.dev

P3 — Voliteľné

Téma	Zdroj
TeaPie — KROS CLI framework na API testovanie	GitHub
Kros.KORM.Extensions.Asp	GitHub
Kros.Ocelot.ETagCaching	GitHub

Praktický cvičný projekt (odporúčaný)

Najefektívnejší spôsob štúdia nie je čítanie dokumentácie, ale **postaviť si vlastný mini-projekt** na rovnakom stacku ako Masha. Po skončení bude vedieť, čo ho v reálnom projekte čaká.

Backend — vlastné API (napr. “Library API”)

1. Klonovať [Kros.AspNetCore.BestPractices](https://github.com/Kros-Software/Kros.AspNetCore.BestPractices), rozbehnúť cez docker-compose (Redis + SQL Server).

2. Postaviť **novú vlastnú službu** (knihy/výpožičky/používatelia, alebo vlastná téma):

- **.NET 10** (nie .NET 8 — cieľová verzia po upgrade Masha)
- ASP.NET Core Web API
- **Kros.KORM** (verejný NuGet)
- **MediatR** pre CQRS
- **FluentValidation** pre validáciu
- **Mapster** pre mapping
- **xUnit + NSubstitute** pre testy

3. Pridať **Azure Function** (timer trigger) — napr. denný export do CSV.

Frontend — Angular SPA napojená na backend

1. `ng new mini-projekt --version=15` — cieľavedome stará verzia (rovnaká ako v Mashe), aby si zažil východiskový stav.
2. **PrimeNG 15 + NgRx 15** (Store + Effects + Entity).
3. **ngrx-forms** pre form state.
4. Jednoduchý CRUD UI napojený na backend z prvej časti.

Skúsiť upgrade na vlastnom projekte

1. Postupne `ng update` na novšie verzie Angularu (16 → 17 → 18 → ...).
2. Sledovať breaking changes na každom kroku.
3. Prerobiť na **standalone components, nový control flow, signals**.
4. Vyskúšať **PrimeNG migráciu** (15 → vyššia, nový Aura theming).
5. Paralelne urobiť aj **.NET upgrade** v backend projekte (8 → 10), riešiť breaking changes Kros.KORM, MediatR a iných knižníc.

Takto si vývojár zažije **oba upgrade procesy v malom merítku** ešte pred tým, ako sa s nimi stretne v Mashe.

AI nástroje vo vývoji

Pozícia je explicitne popísaná ako *“.NET + Angular developer, ktorý **pomocou AI** bude rozširovať funkcionality Masha”*. AI nástroje sú v KROS aktívne používané a očakáva sa od vývojára, že ich bude vedieť efektívne využívať.

Hlavné AI nástroje, s ktorými sa stretne

Nástroj	Použitie	Zdroj
Claude Code (Anthropic)	CLI / IDE agent — generovanie kódu, refactoring, vysvetľovanie kódu, automatizácia migrácií, code review. Hlavný interný nástroj.	claude.com/claude-code
Anthropic API / Claude	Web chat pre dlhšie debatu, brainstorming, písanie dokumentácie	claude.ai

Prečo AI nástroje pri uprade projektoch

Upgrade Angular 15 → 21 aj .NET 8 → 10 sú typy úloh, **kde AI výrazne zrýchľuje prácu:**

- Generovanie migrácií šablón (napr. `*ngIf` → `@if`, `*ngFor` → `@for`)
- Prepis NgModules na standalone components
- Identifikácia breaking changes v knižniciach
- Návrh opráv pri kompilačných chybách po bumpe verzie
- Generovanie unit testov

Claude Code — skills a plugins

Claude Code podporuje **plugins** so **skills** (špecializované workflows). KROS interne používa niekoľko skill balíkov.

Plugin `superpowers` — všeobecné developer workflows

Univerzálny balík skillov pre disciplinovanú prácu. Najpoužívanejšie:

Skill	Použitie
<code>superpowers:brainstorming</code>	Pred začatím akejkoľvek kreatívnej práce (nová feature, komponent, zmena správania) — preskúma zámer, požiadavky a dizajn predtým, než sa píše kód
<code>superpowers:writing-plans</code>	Napísanie implementačného plánu zo špecifikácie pred dotykom kódu
<code>superpowers:executing-plans</code>	Vykonanie hotového plánu v samostatnej session s review checkpointami
<code>superpowers:test-driven-development</code>	TDD postup pri implementácii feature alebo bugfixu
<code>superpowers:systematic-debugging</code>	Systematické debugovanie pri bugoch a test failoch
<code>superpowers:verification-before-completion</code>	Overenie, že práca naozaj funguje, pred PR / claim "hotovo"
<code>superpowers:requesting-code-review</code>	Vyžiadanie code review pred mergom
<code>superpowers:receiving-code-review</code>	Spracovanie feedbacku z code review
<code>superpowers:using-git-worktrees</code>	Izolovaný workspace pre prácu na feature
<code>superpowers:dispatching-parallel-agents</code>	Paralelizácia nezávislých úloh
<code>superpowers:finishing-a-development-branch</code>	Štruktúrované dokončenie vetvy (merge, PR, cleanup)

Odporúčania pre vývojára

1. **Nainštalovať si Claude Code** (alebo aspoň vyskúšať claude.ai cez web) ešte pred nástupom. Pri cvičnom projekte (mini "Library API" + Angular SPA) ho používať aktívne — generovanie controllerov, NgRx slice-ov, testov.
2. **Naučiť sa brainstorming workflow** — pred akoukoľvek väčšou zmenou si s AI premyslieť zámer a dizajn. Ušetrí veľa zlej cesty.
3. **AI nie je auto-pilot** — generovaný kód treba **vždy prečítať, pochopiť a otestovať**. Zodpovednosť za zmergený kód je vždy na vývojárovi.
4. **Pri upgrade projektoch je AI veľmi silný spojenec** — vie spoľahlivo prerobiť stovky súborov na nový control flow alebo standalone components, čo by manuálne trvalo dni.

Po nástupe (až mu pridáme prístupy)

Tieto kroky nemá zmysel pripravovať pred zmluvou, ale je dobré, aby vedel, čo ho čaká:

1. **Onboarding na KROS prostredie** — Azure DevOps, dev VM, prístup do interných NuGet feedov (`nuget.kros.wtf`, `pkgs.dev.azure.com/krossk`), `@kros-sk/*` npm balíky.
2. **Mentor mu prejde štruktúru Masha v páre** — orientácia v repozitári, build a deploy flow.
3. **Prvý jednoduchý ticket** — typu "oprav text v UI" alebo "pridaj jedno pole do existujúceho endpointu". Zažije celý cyklus PR → review → merge → deploy.

4. **Postupne väčšie úlohy** a participácia na uprade projektoch (najprv .NET, potom Angular).

Odporúčané poradie samotných upgradeov

Pre úspech projektu je rozumné:

1. **Najprv .NET upgrade** (8 → 10) — rýchla výhra, naučí workflow s upgrade PR a regresnými testami. Typicky úprava `TargetFramework` + oprava niekoľkých breaking changes v Kros.* knižniciach.
2. **Potom Angular upgrade** (15 → 20/21) — dlhší a náročnejší. Šesť major krokov, každý s vlastným hlavolamom (PrimeNG, ngrx-forms, build system, control flow, signals).

Zhrnutie kľúčových verejných zdrojov

KROS open-source (github.com/Kros-sk):

- [Kros.AspNetCore.BestPractices](#)
- [Kros.KORM](#)
- [Kros.AspNetCore](#)
- [Kros.Utils](#)

Oficiálna dokumentácia:

- [Microsoft Learn — .NET, C#, ASP.NET Core](#)
- [Angular + Angular Update Guide](#)
- [NgRx](#)
- [RxJS](#)
- [TypeScript](#)
- [PrimeNG](#)

Knižnice tretích strán používané v Mashe:

- [MediatR](#)
- [FluentValidation](#)
- [Mapster](#)
- [Refit](#)
- [Polly](#)
- [Flurl](#)
- [EPPlus](#)